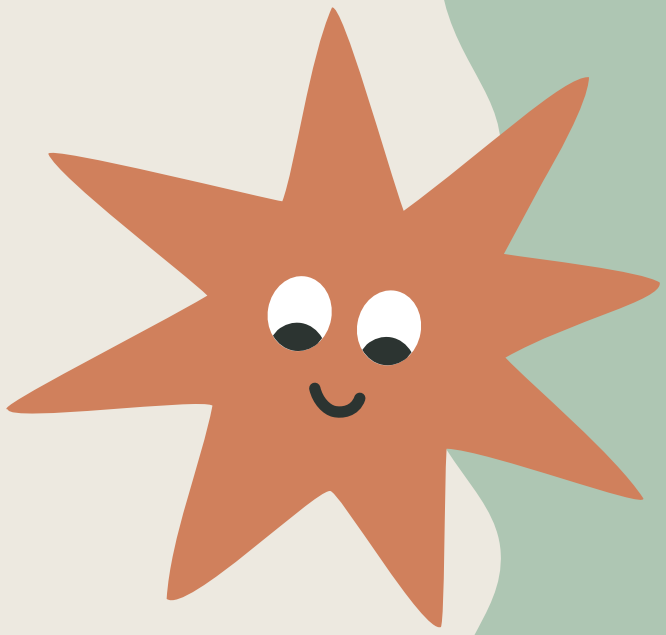
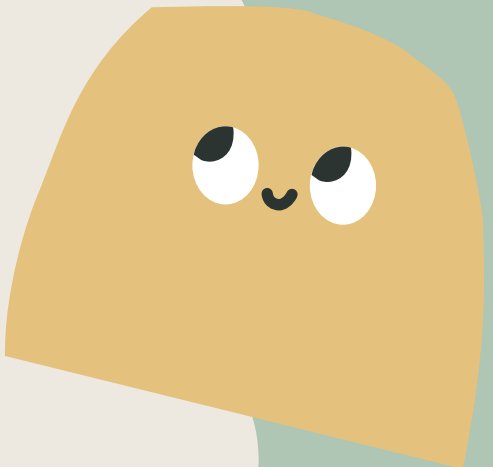
Breadth-First Search (BFS) in Artificial Intelligence

Lecture 10





Recap: Search in AI

- In Artificial Intelligence, search is a problem-solving technique in which an agent explores possible states to find a path from an initial state to a goal state
- We have already learned that search strategies are divided into two major categories:
  - Uninformed Search
  - Informed Search
- Today, we will study one of the most important uninformed search algorithms: Breadth-First Search (BFS)
- BFS does not use heuristic information. It explores the search space according to the depth of nodes

Prepared By: Ehtisham Rasheed

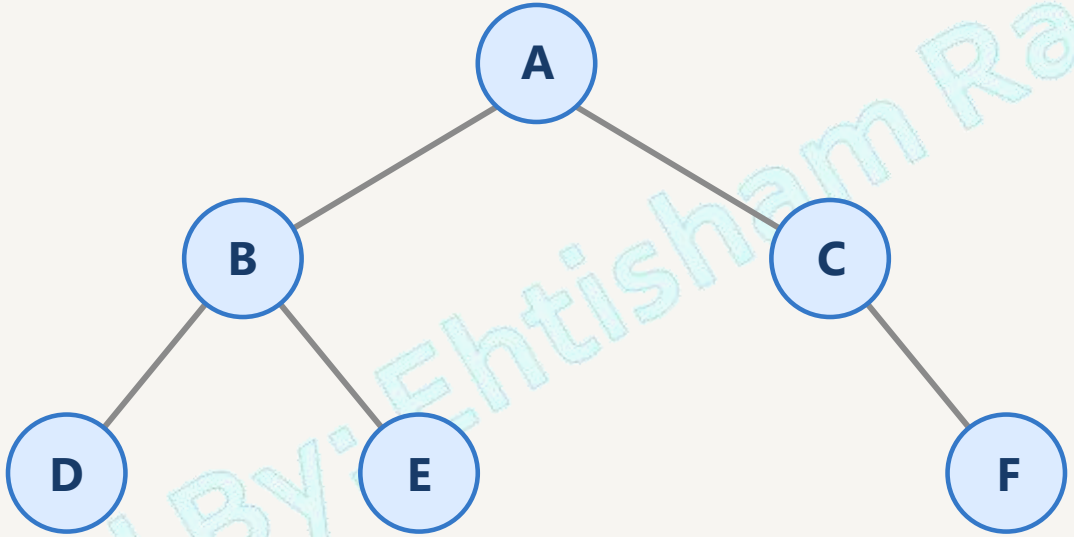
What Is Breadth-First Search?

- Breadth-First Search (BFS) is an uninformed search algorithm that explores all nodes at the current depth before moving to nodes at the next depth.
- It systematically explores a state space **level by level**, visiting all neighbor nodes at the current depth before moving deeper into the structure
- Also known as a **blind** or **brute-force** search

Prepared By. Ehtisham Rasheed

What Is Breadth-First Search?

➢Explore shallow nodes before deeper nodes

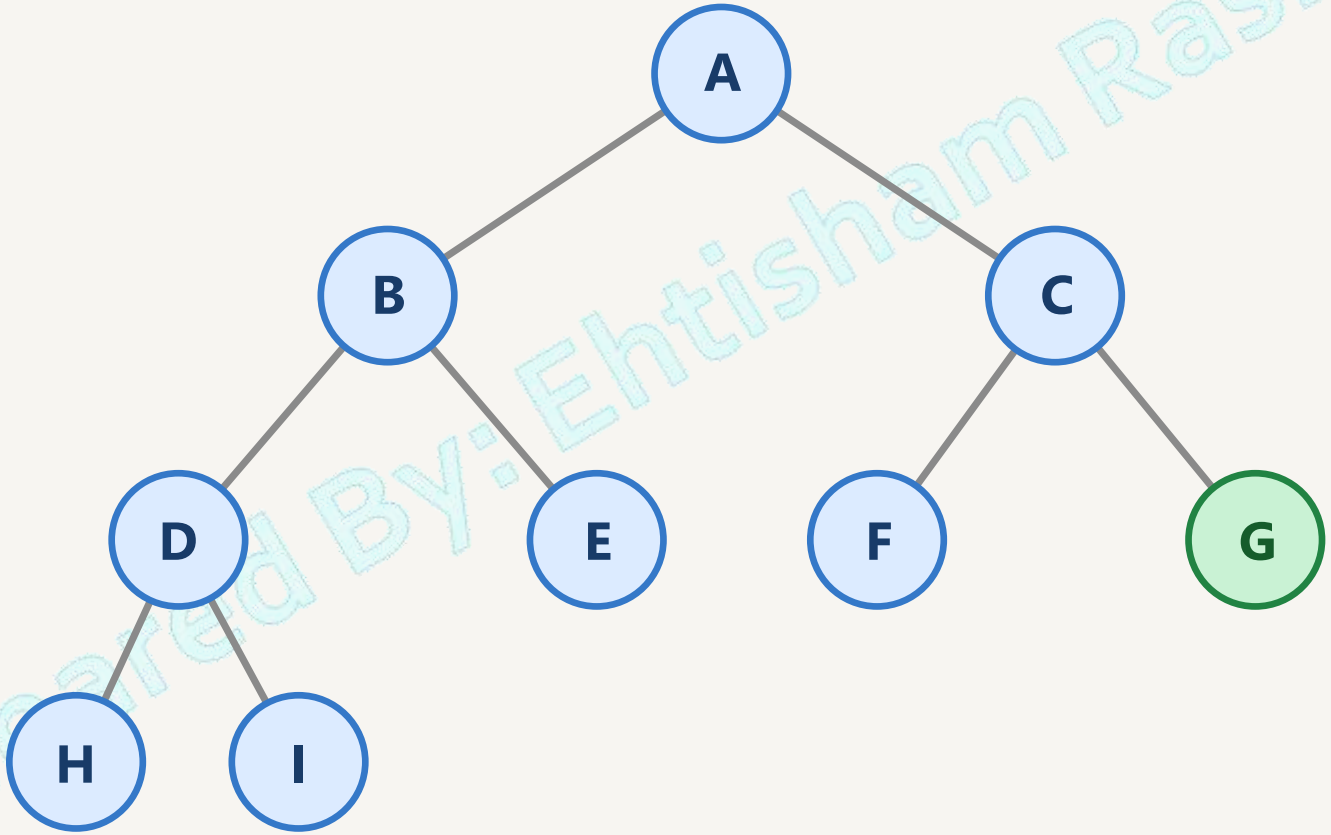


➢BFS visits A → B → C → D → E → F (left-to-right order)

Prepared By: Ehtisham Rasheed

Understanding BFS Through an Example

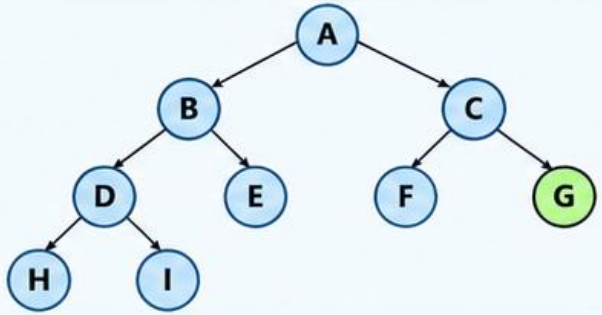
➢The goal is to find node **G**, starting from node **A**



Prepared By: Ehtisham Rasheed

Breadth-First Search (BFS) – Step by Step Example (Tree)

Given Tree



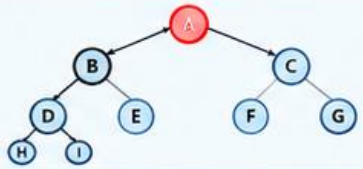
Adjacency List (Left to Right Order)

Node	Children (Left to Right)
A	B, C
B	D, E
C	F, G
D	H, I
E	–
F	–
G	–
H	–
I	–

BFS Rules

1. Start from the root node (A).
2. Use a queue (FIFO).
3. Mark a node as visited when it is added to the queue.
4. Consider children from left to right (as shown in the tree).
5. Do not add a child if it is already visited.

Step 1: Start with A

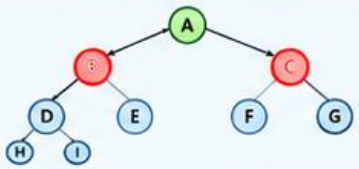


Action: Add **A** to queue and mark as visited.

Queue: [A]

Visited: { A }

Step 2: Expand A

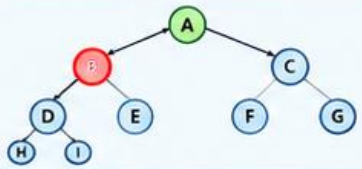


Action: Remove **A** from queue. Add children **B, C**.

Queue: [B, C]

Visited: { A, B, C }

Step 3: Expand B

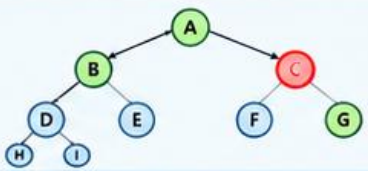


Action: Remove **B** from queue. Add children **D, E**.

Queue: [C, D, E]

Visited: { A, B, C, D, E }

Step 4: Expand C

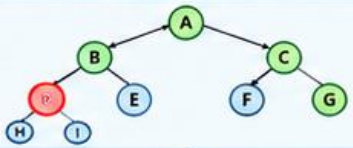


Action: Remove **C** from queue. Add children **F, G**.

Queue: [D, E, F, G]

Visited: { A, B, C, D, E, F, G }

Step 5: Expand D

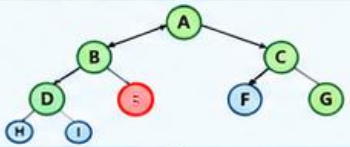


Action: Remove **D** from queue. Add children **H, I**.

Queue: [E, F, G, H, I]

Visited: { A, B, C, D, E, F, G, H, I }

Step 6: Expand E

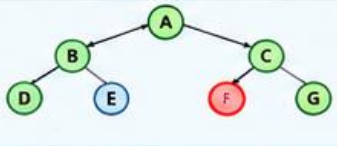


Action: Remove **E** from queue. No new node (leaf node).

Queue: [F, G, H, I]

Visited: { A, B, C, D, E, F, G, H, I }

Step 7: Expand F

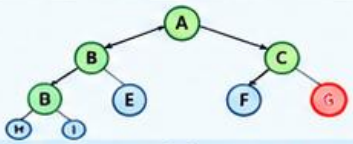


Action: Remove **F** from queue. No new node (leaf node).

Queue: [G, H, I]

Visited: { A, B, C, D, E, F, G, H, I }

Step 8: Expand G



Action: Remove **G** from queue. No new node (leaf node).

Queue: [H, I]

Visited: { A, B, C, D, E, F, G, H, I }

Step 9: Expand H



Action: Remove **H** from queue. No new node (leaf node).

Queue: [I]

Visited: { A, B, C, D, E, F, G, H, I }

Step 10: Expand I

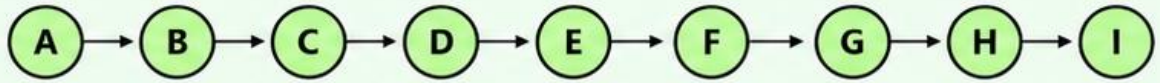


Action: Remove **I** from queue. No new node (leaf node).

Queue: [ ] (Empty)

Visited: { A, B, C, D, E, F, G, H, I }

BFS Traversal Order



BFS Traversal — Step by Step

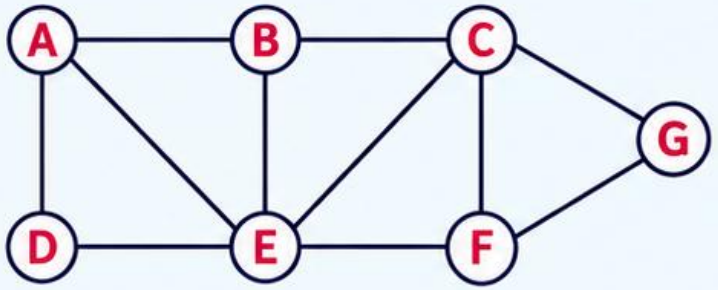
Step	Node selected	Frontier after processing
1	A	B, C
2	B	C, D, E
3	C	D, E, F, G
4	D	E, F, G, H, I
5	E	F, G, H, I
6	F	G, H, I
7	G	Goal found - stop

- BFS traversal order until the goal is found:
- A → B → C → D → E → F → G

Prepared by

Breadth-First Search (BFS) – Step by Step Example (Adjacency in Alphabetical Order)

Given Graph



Adjacency List (Alphabetical Order)

Node	Neighbors (sorted alphabetically)
A	B, D, E
B	A, C, E
C	B, E, F, G
D	A, E
E	A, B, C, D, F
F	C, E, G
G	C, F

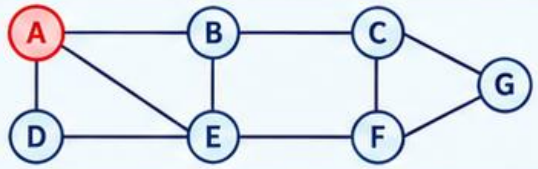
BFS Rules

1. Start from the given node (A).
2. Use a queue (FIFO).
3. Mark a node as visited when it is added to the queue.
4. Consider neighbors in **alphabetical order**.
5. Do not add a neighbor if it is already visited.

BFS Traversal Order



Step 1: Start with A

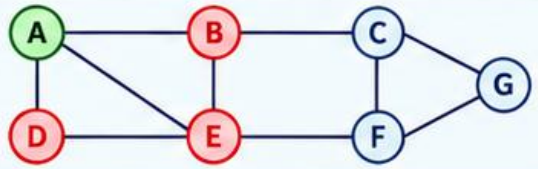


Action: Add A to queue and mark as visited.

Queue: [A]

Visited: { A }

Step 2: Expand A

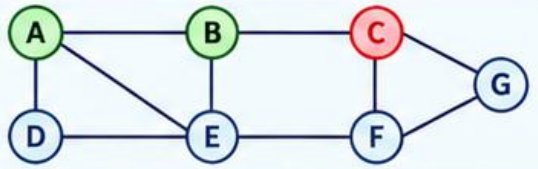


Action: Remove A from queue. Add neighbors B, D, E (alphabetical).

Queue: [B, D, E]

Visited: { A, B, D, E }

Step 3: Expand B

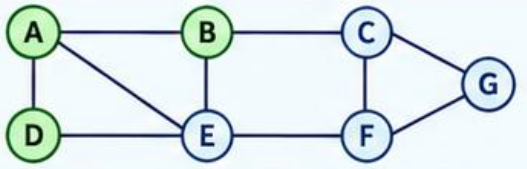


Action: Remove B from queue. Add unvisited neighbor C.

Queue: [D, E, C]

Visited: { A, B, D, E, C }

Step 4: Expand D

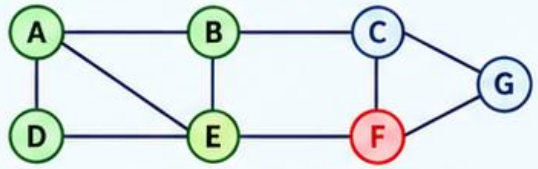


Action: Remove D from queue. Its neighbors (A, E) are already visited.

Queue: [E, C]

Visited: { A, B, D, E, C }

Step 5: Expand E

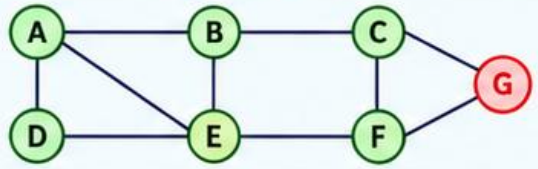


Action: Remove E from queue. Add unvisited neighbor F.

Queue: [C, F]

Visited: { A, B, D, E, C, F }

Step 6: Expand C

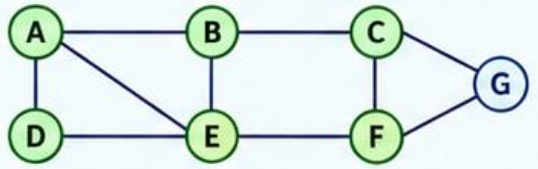


Action: Remove C from queue. Add unvisited neighbor G.

Queue: [F, G]

Visited: { A, B, D, E, C, F, G }

Step 7: Expand F

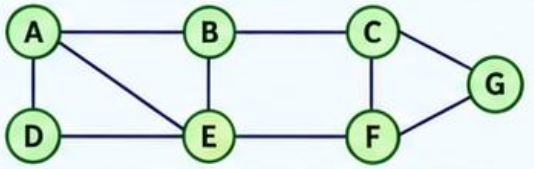


Action: Remove F from queue. Its neighbors (C, E, G) are already visited.

Queue: [G]

Visited: { A, B, D, E, C, F, G }

Step 8: Expand G



Action: Remove G from queue. Its neighbors (C, F) are already visited.

Queue: [ ] (Empty)

Visited: { A, B, D, E, C, F, G }

Traversal Order vs. Solution Path

- Don't confuse with Traversal Order with Solution Path
- Using our example:
  - Traversal order: A → B → D → E → C → F → G
  - Solution path: A → B → C → G
- The traversal order shows the nodes BFS explored before reaching the goal
- The solution path shows only the sequence of nodes from the initial state to the goal
- BFS may explore several nodes that are not part of the final solution path

Prepared By: Ehtisham Rasheed

How Does BFS Find the Solution Path?

- BFS maintains a **Parent** for every newly discovered node
- Parent = the node from which the node was first discovered

Node	Parent
B	A
D	A
E	A
C	B
F	E
G	C

- Solution Path = A → B → C → G

Prepashed

BFS Algorithm (Logical Pseudocode)

```
BFS(start, goal):

    Create an empty queue
    Add start to the queue

    Mark start as visited

    While queue is not empty:

        Remove the front node from queue

        If node is the goal:
            Return solution

        For each unvisited neighbor of node:

            Mark neighbor as visited
            Store its parent
            Add neighbor to the queue

    Return failure
```

Prepar
sheed

Traversal Order vs. Solution Path

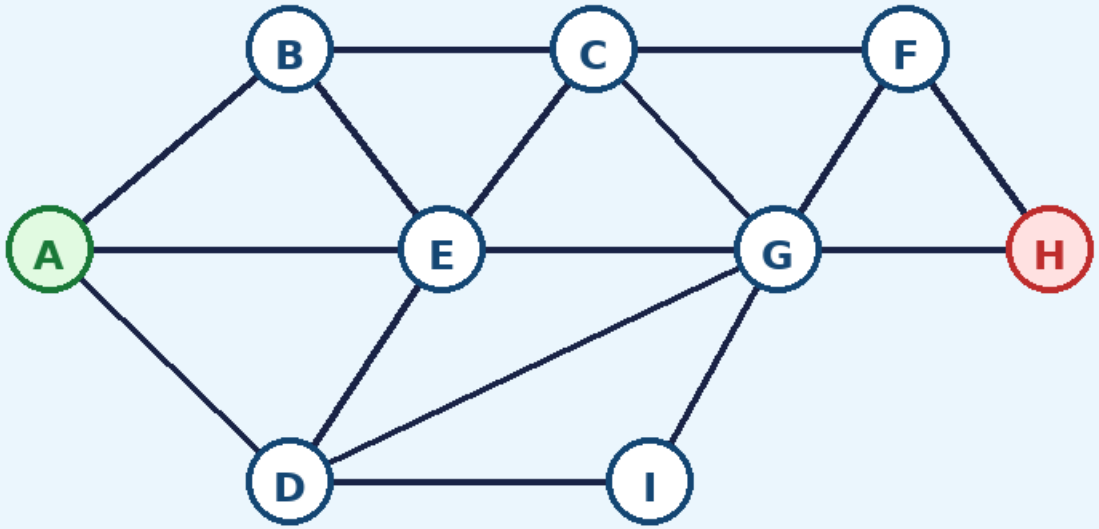
- Students often confuse the BFS traversal order with the solution path
- Using our example:
  - Traversal order: A → B → C → D → E → F → G
  - Solution path: A → C → G
- The traversal order shows the nodes BFS explored before reaching the goal
- The solution path shows only the sequence of nodes from the initial state to the goal
- BFS may explore several nodes that are not part of the final solution path

Prepared By: Ehtisham Rasheed

BFS Practice Problems — With Answers

Graph 1

Start: A Goal: H



BFS Traversal Order (from A):

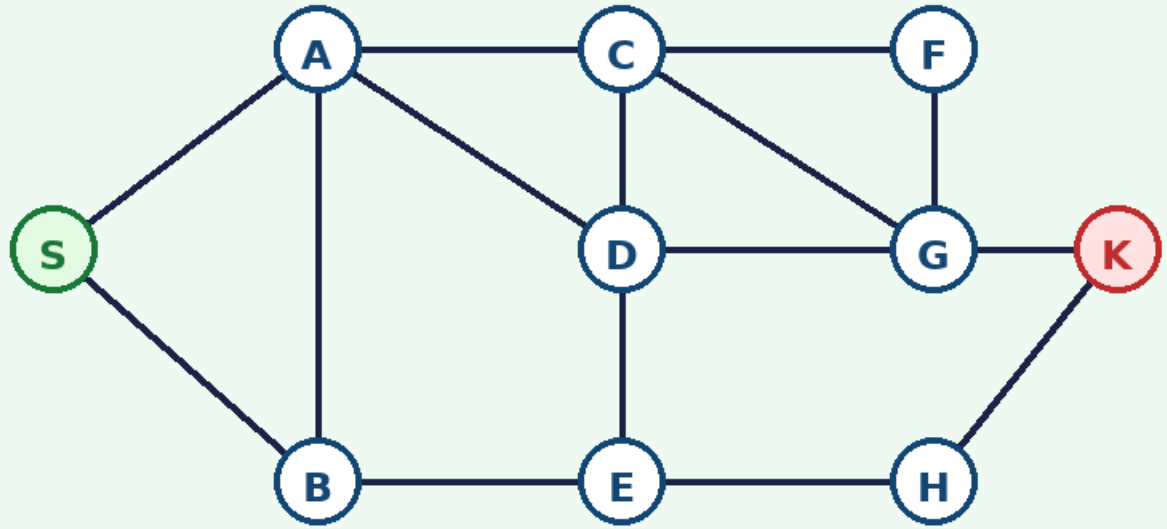
A → B → D → E → C → G → I → F → H

BFS Solution Path (A to H):

A → D → G → H

Graph 2

Start: S Goal: K



BFS Traversal Order (from S):

S → A → B → C → D → E → F → G → H → K

BFS Solution Path (S to K):

S → A → C → G → K

Rule: Use a FIFO queue and consider neighbors in alphabetical order. Goal test when a node is removed from the queue.

BFS Properties

- Is BFS Complete?
  - Yes, BFS is complete when the branching factor is finite
- Is BFS Optimal?
  - BFS is optimal when all actions have equal cost
- Time and Space Complexity
  - Let **b** = branching factor and **d** = depth of the shallowest goal
- BFS has the following commonly taught worst-case complexities:
  - Time Tree = $O(b^{d+1})$ Graph = $O(V + E)$
  - Space Tree = $O(b^{d+1})$ Graph = $O(V)$

Prepared By: Ehtisham Rasheed

Advantages and Disadvantages

- Advantages
  - Simple and systematic
  - Complete under finite branching
  - Finds the shallowest solution
  - Optimal when all steps are equal
  - Does not require heuristic knowledge
- Disadvantages
  - Can consume a large amount of memory
  - May explore many nodes before reaching the goal
  - Can be slow when the goal is very deep
  - Not optimal when step costs vary

Prepared By: Entisham Rasheed

Practical AI Applications

- Game AI & Puzzle Solving
  - Used to find the absolute minimum moves required to solve puzzles like the 8-puzzle, Rubik's cube, or for basic NPC pathfinding in mazes
- Web Crawling
  - Search engines use BFS to systematically index web pages by discovering links layer by layer from an initial seed page
- Social Network Analysis
  - Calculating the "degrees of separation" or finding mutual friends between users on platforms like LinkedIn or Meta
- Routing and Network Broadcasting
  - Ensuring efficient data packet distribution across local nodes or finding neighboring points in GPS navigation

Prepared By: Ehtisham Rasheed